3.6 Final classes and methods

Chapter 19 ends with two ways to limit extension. A **final class** cannot have subclasses. A **final method** cannot be overridden, although its class can still have subclasses.

Protect one implementation

Run this complete example in `3-6-final-method.php`:

```php
<?php
declare(strict_types=1);

class Product
{
    private int $skuId;

    public function __construct(int $skuId)
    {
        $this->setSkuId($skuId);
    }

    final public function setSkuId(int $skuId): void
    {
        if ($skuId <= 0) {
            throw new InvalidArgumentException('SKU must be positive.');
        }

        $this->skuId = $skuId;
    }

    final public function getSkuId(): int
    {
        return $this->skuId;
    }
}

class Book extends Product
{
}

$book = new Book(101);
echo $book->getSkuId(), PHP_EOL;
$book->setSkuId(202);
echo $book->getSkuId(), PHP_EOL;
```

Output: `101`, then `202` on separate lines. The inherited final methods can still be **called**.

In a separate copy, add `public function setSkuId(int $skuId): void {}` inside Book. PHP rejects overriding the final method. Remove it. Calling `new Book(0)` instead throws the validation exception at

runtime.

This version deliberately keeps the SKU private and routes construction through validation. `final` by itself would not prevent a subclass from writing directly to protected state. Nor does this design prevent a child from defining its own constructor that omits initialization. Subclasses cannot replace these methods. It is not a general security boundary or a complete proof of object validity.

Close the whole class

In another copy, change `class Product` to `final class Product`. Loading the Book declaration now fails because Book attempts to extend Product. You can still instantiate Product directly. A final class does not make its objects immutable; its methods may continue to update state.

Choose deliberately

Design intention	**Possible choice**
Only specialized objects should exist	Abstract class
Subclasses may specialize some behavior	Ordinary class and overridable methods
One operation must retain this implementation	Final method
Subclassing is not a supported extension point	Final class

Final can limit compatibility obligations for a library author, but it also limits customization. Explain the reason for the restriction. Avoid presenting it as a universal default or treating every inheritance hierarchy as beneficial.